Evolução da arquitetura — Retena (Parte 4)

Diagrama atualizado: `08_arquitetura_executada.png` (também em `../../02_Diagramas/`). Ele substitui a visão "desenhada" da Fase 6 (`06_arquitetura_oracle.png`) por aquilo que foi executado de fato, e marca o que ainda é runbook.

1. Decisões técnicas tomadas na prática

Decisão	Na Fase 6 (planejado)	Agora (executado)	Motivo
Onde o modelo roda	Oracle Machine Learning in-database	Executado: `DBMS_DATA_MINING` com Random Forest e GLM no Oracle AI Database 26ai Free; três modelos criados em 7 s	sem servidor de ML, dado não sai do banco
Como o risco é servido	view calculada ao vivo	**Mudou:** tabela `RISCO_ALUNO_SNAPSHOT` atualizada por `PRC_ATUALIZAR_RISCO` (MERGE idempotente) via `DBMS_SCHEDULER` toda segunda 06:00	a view ao vivo levava mais de 60 s; o snapshot responde na hora e dá histórico
Features	SQL no banco	Executado: `02_features_transicao.sql` e `03_features_semanais.sql` (89 s); **ajuste de desenho:** janela móvel de 28 dias e partição por semana na Fase 7	crescimento linear com o volume
Carga	Object Storage + `DBMS_CLOUD`	Executado localmente com python-oracledb (72–146 s); Object Storage fica como runbook	sem conta OCI na Fase 6
Exposição para o painel	views JSON para APEX	Executado: `VW_RISCO_JSON`, Duality View `RISCO_ALUNO_DV`, consultas de `07_apex_ready.sql`; painel atual é HTML estático publicado	APEX depende do Autonomous
Registro do contato	"em um clique"	Protótipo no navegador (localStorage); **decisão:** tabela `CONTATO_RESULTADO` + ORDS na Fase 7	validar o fluxo com usuários antes de persistir
Modelo de referência	pipeline Python	Mantido como avaliação (benchmark de 6 configurações, OULAD); **decisão:** features só em SQL na Fase 7	evitar duas implementações divergentes
Operação	—	scripts `dia_da_banca/`: preparação em 1 min, scoring ao vivo em 25 s, servidor local de reserva	demonstrações sem depender de internet

2. Integrações reais

- **Oracle AI Database 26ai Free (Docker)**: `EVENTOS_LMS` (684.723 linhas), tabelas de features, três modelos OML, `RISCO_ALUNO_SNAPSHOT`, views JSON, Duality View, job semanal. Conexão `python-oracledb` thin (`STARTUP@localhost:1521/FREEPDB1`).
- **Pipeline Python** (`05_MVP`): lê o mesmo export, gera o dashboard HTML e as figuras; validação temporal e OULAD.
- **GitHub Pages**: publica landing, dashboard, evidências e diagramas (`lucashenklain.github.io/retena`).
- **OCI (runbook executado em parte)**: conexão dual com wallet, exportação CSV gzip (dry-run), MERGE do snapshot testado duas vezes; `DBMS_CLOUD.COPY_DATA`, Object Storage, APEX e Select AI aguardam a conta.

3. Como a arquitetura suporta crescimento

1. **Um schema por instituição no Autonomous AI Database Serverless.** Isolamento de dados (LGPD) e auto-scaling de CPU por carga; a IES nova nasce com o modelo de referência e passa a usar o próprio após 8 semanas de dados.
2. **Volume.** O custo de cada etapa é linear nos eventos: carga em lotes, features por janela de 28 dias (em vez do histórico inteiro), partição semanal de `EVENTOS_LMS` e dos agregados. Para 12 mil alunos a extrapolação do job semanal fica abaixo de 10 minutos com a janela móvel (hoje seriam ~90 min recalculando tudo).
3. **Ingestão.** Entrada por CSV padrão do LMS no Object Storage com `DBMS_CLOUD.COPY_DATA`; validação por contrato na carga; quarentena de linhas inválidas. Depois, API quando o LMS tiver.
4. **Consumo.** APEX e ORDS leem as mesmas views JSON; o dashboard HTML, a landing e o relatório PDF consomem o mesmo snapshot, então não há dois cálculos para conciliar.
5. **Modelos.** Versionados no banco (`USER_MINING_MODELS`), retreino mensal pelo job e avaliação automática no teste temporal; o modelo só é promovido se a AUC não cair abaixo de 0,80 (limite inferior já observado nos cortes).

4. Como garante escalabilidade e manutenção

Preocupação	Mecanismo
Reprodutibilidade	tudo em SQL numerado (01–07) e scripts Python; reexecução completa em ~7 min com logs; `docker compose` para o ambiente local
Observabilidade	logs por script com contagem de passos e erros; histórico do job em `USER_SCHEDULER_JOB_RUN_DETAILS`; `DT_SCORING` em cada linha do snapshot
Idempotência	MERGE por (aluno, corte): rodar duas vezes não duplica (testado 2×)
Segurança	usuário de aplicação separado de SYSTEM; wallet no Autonomous; TDE e auditoria nativos; pseudonimização fora do painel
Evolução sem quebrar	views JSON como contrato entre banco e painel; dashboard e APEX trocam sem mexer no modelo
Custo	Always Free para piloto (1 OCPU, 20 GB); custo marginal por aluno próximo de zero; margem bruta alvo de 80%
Manutenção do modelo	retreino mensal, monitor de AUC por corte, rollback para a versão anterior do modelo se a métrica cair

5. O que ainda é desenho (e quando vira execução)

Componente	Estado	Fase 7
Autonomous AI Database Serverless	runbook de 60 min	provisionar Always Free e executar 01–07
Object Storage + `DBMS_CLOUD.COPY_DATA`	runbook	ingestão do CSV padrão
APEX (painel) e ORDS (API de registro)	consultas prontas no banco	6 telas do protótipo
Select AI	roadmap	perguntas em português sobre a base
Features incrementais e partição	desenho neste relatório	teste de carga com 12 mil alunos simulados